Efficient Dependency Resolution in IFC-aware Decentralized Programming

Steffan Christ Sølvsten and Aslan Askarov

PRiSC 2026

AARHUS UNIVERSITY

Module systems are difficult!

$\boxed{\mathtt{A}}$ $\boxed{\mathtt{B}}$

[1] Using hashes to uniquely identify an object/class is called a "*fingerprint*" in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

$$\boxed{\texttt{A}} \xrightarrow{\quad f_{\texttt{Mod}} \quad} \boxed{\texttt{B}}$$

[1] Using hashes to uniquely identify an object/class is called a "*fingerprint*" in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

$$
\begin{tikzcd}
\boxed{\mathtt{A}} \arrow[rrr, "f_{\mathtt{Mod}}"] \arrow[rrr, bend right, "g_{\mathtt{Mod}}"'] & & & \boxed{\mathtt{B}}
\end{tikzcd}
$$

[1] Using hashes to uniquely identify an object/class is called a "*fingerprint*" in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

How can one add a module cache?

How can one add a ~~module~~ *dependency* cache?

Leaky Cache[2] (Confidentiality)

$\boxed{\texttt{A}}$ $\boxed{\texttt{C}}$ $\boxed{\texttt{B}}$

[2]This behaviour is a remanifestation of a "*read channel*" in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Leaky Cache[2] (Confidentiality)

$$\begin{tikzcd} \boxed{\mathtt{A}} \arrow[r, "f_{\mathtt{Mod}}"] & \boxed{\mathtt{C}} & \boxed{\mathtt{B}} \end{tikzcd}$$

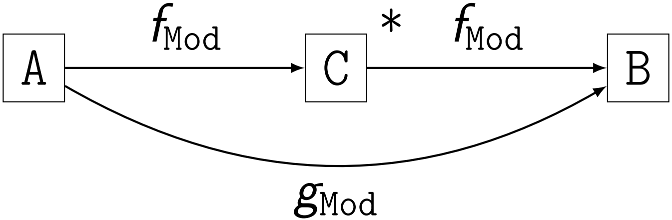
[2]This behaviour is a remanifestation of a "*read channel*" in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Leaky Cache[2] (Confidentiality)

$$\boxed{\mathtt{A}} \xrightarrow{f_{\mathtt{Mod}}} \boxed{\mathtt{C}}^{\,*} \xrightarrow{f_{\mathtt{Mod}}} \boxed{\mathtt{B}}$$

[2]This behaviour is a remanifestation of a "*read channel*" in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Leaky Cache[2] (Confidentiality)



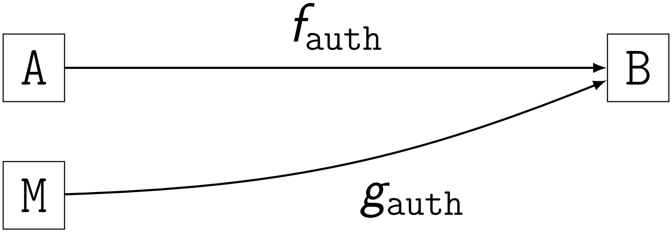
[2] This behaviour is a remanifestation of a "*read channel*" in
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Replay Dependency[3] (Integrity)

$$\boxed{\texttt{A}} \xrightarrow{f_{\texttt{auth}}} \boxed{\texttt{B}}$$

[3] This behavior is a remanifestation of the "*provider-bounded authority*" requirement in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Replay Dependency[3] (Integrity)



[3] This behavior is a remanifestation of the "*provider-bounded authority*" requirement in O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012).

Chain of Death[4] (Availability)

$\boxed{\mathtt{A}}$

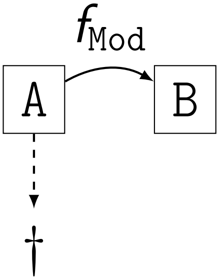
[4] This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)

$$\boxed{\texttt{A}} \xrightarrow{f_{\texttt{Mod}}} \boxed{\texttt{B}}$$

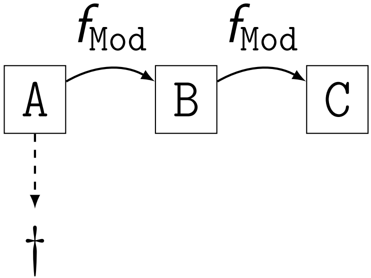
[4]This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



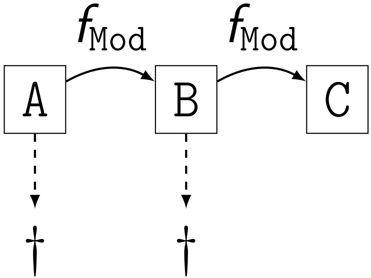
[4] This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



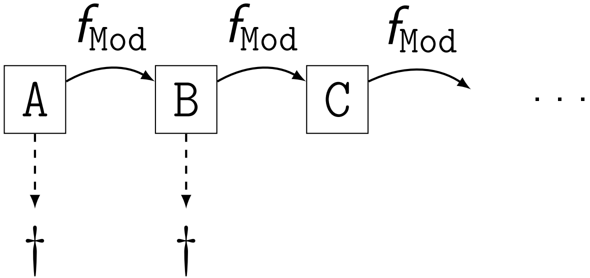
[4]This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



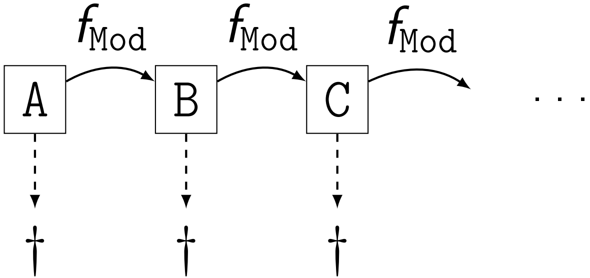
[4]This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



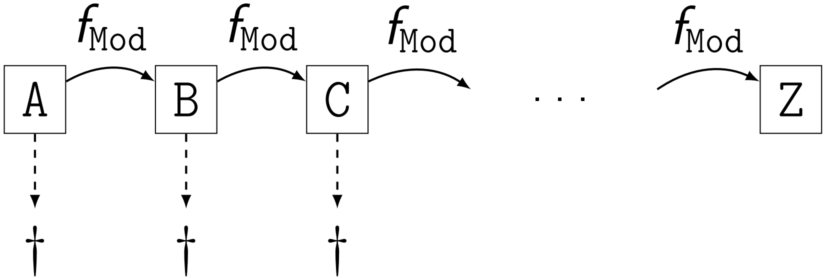
[4]This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



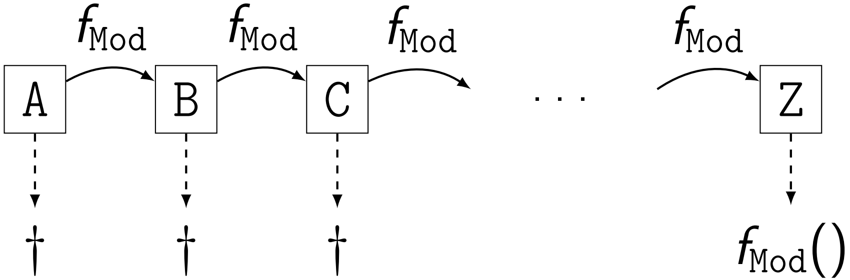
[4]This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*.
O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



[4] This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*. O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

Chain of Death[4] (Availability)



[4] This is related to the notion of "*mobile code*". But, code should outlive its origin, i.e. be *nomadic*. O. Arden et al. *Sharing Mobile Code Securely with Information Flow Control* (2012)

How can one add an **IFC-aware** cache?

Troupe Programming Language

```
(* Alice.trp *)
let val msg = "Hello, Bob!"
    fun f () = print msg
in send (@bob, f)
end
```

```
(* Bob.trp *)
let fun loop () =
    receive [hn f => spawn f];
    loop ()
in loop ()
end
```

- **Decentralized Programming**
  For creation of open self-organizing distributed systems.
- **Information Flow Control**
  Ensures via a monitor that secret information is not by accident shared with nor affected by unwanted parties.
- **Dynamically Typed**
  Permits the dynamic behaviour needed by a truly open and updateable system.

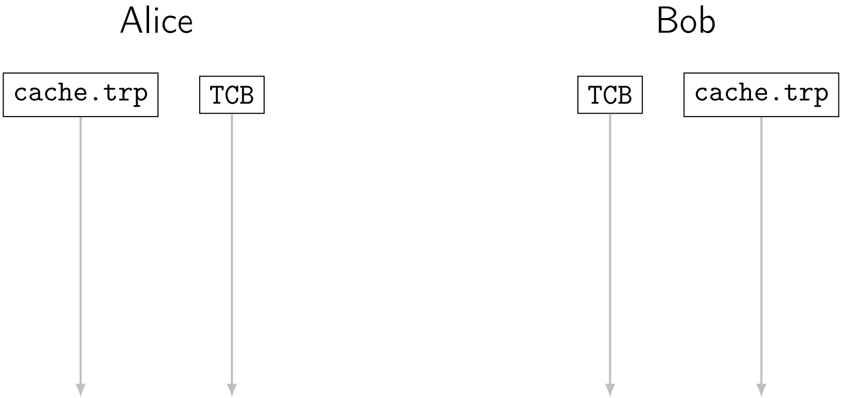
How can one add an **IFC-aware** cache?

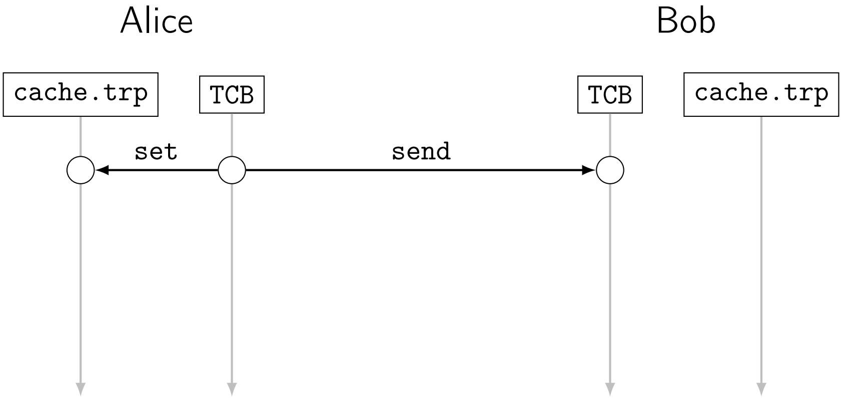
How can one add an **IFC-aware** cache?

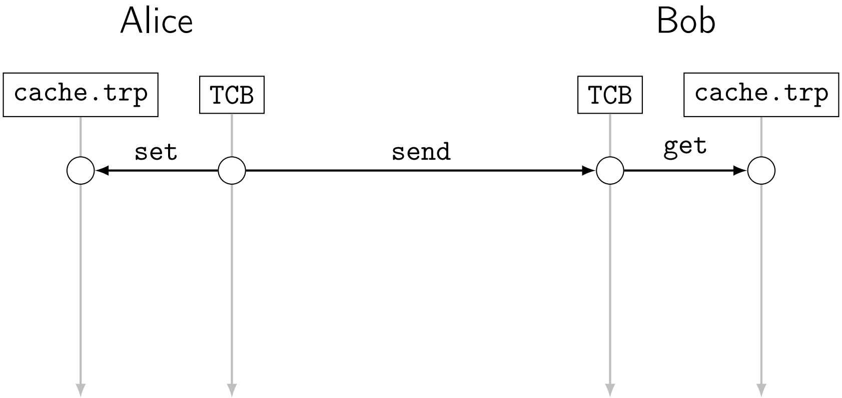
(with the minimal extension to the TCB...)

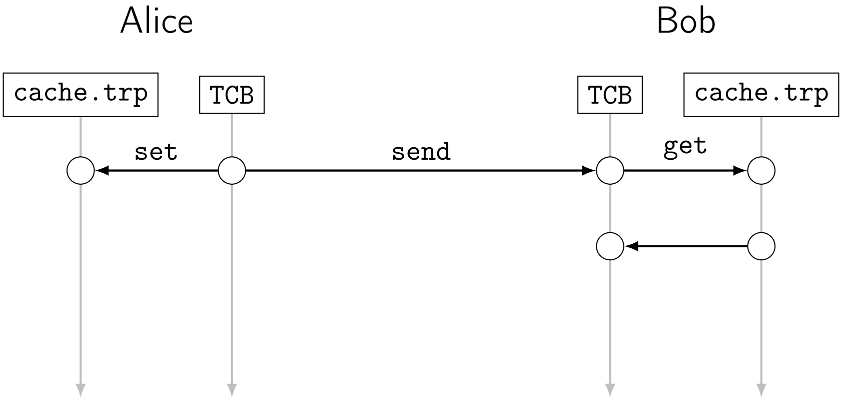
How can one add an IFC-aware cache?

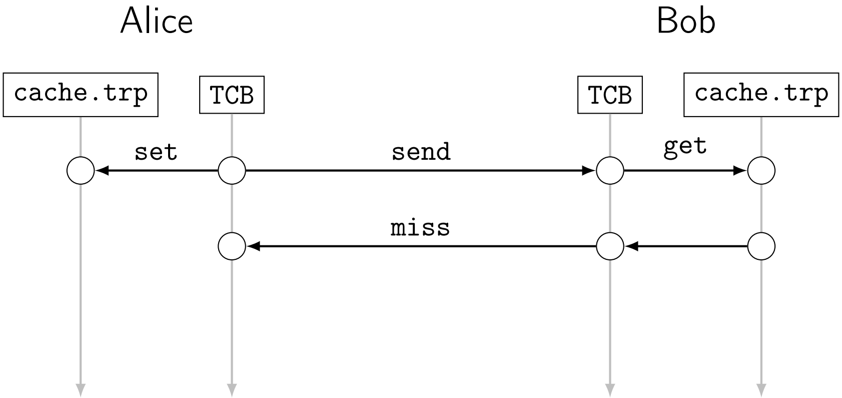
(by using Troupe itself. . . ?)

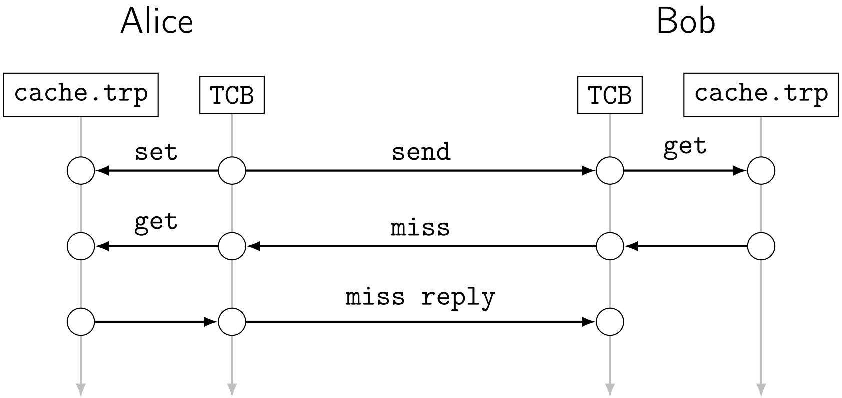

Alice
Bob
cache.trp
TCB
TCB
cache.trp

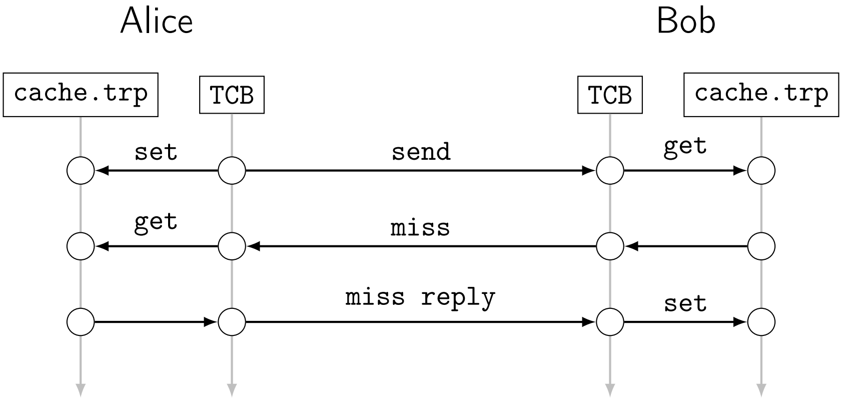

Alice
Bob
cache.trp
TCB
TCB
cache.trp
set
send


Alice
Bob
cache.trp
TCB
TCB
cache.trp
set
send
get


Alice
Bob
cache.trp
TCB
TCB
cache.trp
set
send
get


Alice
Bob
cache.trp
TCB
TCB
cache.trp
set
send
get
miss


Alice
Bob
cache.trp
TCB
TCB
cache.trp
set
send
get
get
miss
miss reply


Alice
Bob
cache.trp
TCB
TCB
cache.trp
set
send
get
get
miss
miss reply
set

Changes to Runtime (TCB)

- **Peer-to-peer layer:**
  Extend with `miss` and `miss reply` messages which are always sent by value.
- **Serialization:**
  Dependency resolution requires introspection of function closures.
  - During `serialize`, identify (sub)values to be referenced by hash.
  - To `deserialize`, (1) extract references (2) deserialize given (all resolved) dependencies.
- **Hash:**
  Hashing Troupe values requires introspection of function closures and source.
- **Scheduler:**
  Extend to run Troupe code from the peer-to-peer layer.

Additions in Troupe: IFC-aware `Map` / `HashMap`

$$C \xrightarrow{f_{\texttt{Mod}}@\{C\}} B \implies \text{miss} \qquad A \xrightarrow{g_{\texttt{Mod}}@\{\}} B \implies \text{miss} \qquad D \xrightarrow{h_{\texttt{Mod}}@\{D\}} B$$

Additions in Troupe: IFC-aware `Map` / `HashMap`

$$C \xrightarrow{f_{\texttt{Mod}}@\{C\}} B \implies \text{miss} \qquad A \xrightarrow{g_{\texttt{Mod}}@\{\}} B \implies \text{miss} \qquad D \xrightarrow{h_{\texttt{Mod}}@\{D\}} B \implies \text{hit}$$

Additions in Troupe: IFC-aware `Map` / `HashMap`

$$C \xrightarrow{f_{\texttt{Mod}}@\{C\}} B \implies \text{miss} \qquad A \xrightarrow{g_{\texttt{Mod}}@\{\}} B \implies \text{miss} \qquad D \xrightarrow{h_{\texttt{Mod}}@\{D\}} B \implies \text{hit}$$

"Specification":

An IFC-aware `HashMap` is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its `insert` and `remove`[5] history:

$$\texttt{i}(x \mapsto 1)\texttt{@\{\}}, \quad \texttt{i}(x \mapsto 2)\texttt{@\{A\}}, \quad \texttt{i}(y \mapsto 3)\texttt{@\{B\}}, \quad \texttt{r}(x)\texttt{@\{B\}}, \quad \texttt{i}(y \mapsto 4)\texttt{@\{\}}, \quad \ldots$$

[5] For this particular use case, we actually only need public (`@{}`) `remove` operations.

Additions in Troupe: IFC-aware `Map` / `HashMap`

$$C \xrightarrow{f_{\texttt{Mod}}\texttt{@}\{C\}} B \implies \text{miss} \qquad A \xrightarrow{g_{\texttt{Mod}}\texttt{@}\{\}} B \implies \text{miss} \qquad D \xrightarrow{h_{\texttt{Mod}}\texttt{@}\{D\}} B \implies \text{hit}$$

"Specification":

An IFC-aware `HashMap` is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its `insert` and `remove`[5] history:

$$\texttt{i}(x \mapsto \texttt{1})\texttt{@\{\}}, \quad \texttt{i}(x \mapsto \texttt{2})\texttt{@\{A\}}, \quad \texttt{i}(y \mapsto \texttt{3})\texttt{@\{B\}}, \quad \texttt{r}(x)\texttt{@\{B\}}, \quad \texttt{i}(y \mapsto \texttt{4})\texttt{@\{\}}, \quad \ldots$$

$$\shortparallel$$

$$\texttt{@\{\}}$$

$$\Downarrow$$

$$[x \mapsto \texttt{1@\{\}},\ y \mapsto \texttt{4@\{\}}]\ \texttt{@\{\}}$$

[5] For this particular use case, we actually only need public (`@{}`) `remove` operations.

Additions in Troupe: IFC-aware `Map` / `HashMap`

$$C \xrightarrow{f_{\texttt{Mod}}\texttt{@}\{C\}} B \implies \text{miss} \qquad A \xrightarrow{g_{\texttt{Mod}}\texttt{@}\{\}} B \implies \text{miss} \qquad D \xrightarrow{h_{\texttt{Mod}}\texttt{@}\{D\}} B \implies \text{hit}$$

"Specification":

An IFC-aware `HashMap` is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its `insert` and `remove`[5] history:

$$\texttt{i}(x \mapsto 1)\texttt{@\{\}},\quad \texttt{i}(x \mapsto 2)\texttt{@\{A\}},\quad \texttt{i}(y \mapsto 3)\texttt{@\{B\}},\quad \texttt{r}(x)\texttt{@\{B\}},\quad \texttt{i}(y \mapsto 4)\texttt{@\{\}},\quad \ldots$$

$$\shortparallel$$

$$\texttt{@\{A\}}$$

$$\Downarrow$$

$$[x \mapsto 2\texttt{@\{A\}},\ y \mapsto 4\texttt{@\{\}}]\ \texttt{@\{A\}}$$

[5] For this particular use case, we actually only need public (`@{}`) `remove` operations.

Additions in Troupe: IFC-aware `Map` / `HashMap`

$$C \xrightarrow{f_{\texttt{Mod}}\texttt{@}\{C\}} B \implies \text{miss} \qquad A \xrightarrow{g_{\texttt{Mod}}\texttt{@}\{\}} B \implies \text{miss} \qquad D \xrightarrow{h_{\texttt{Mod}}\texttt{@}\{D\}} B \implies \text{hit}$$

"Specification":

An IFC-aware `HashMap` is a single data structure with public and secret key-value pairs which provides a IFC-aware projections of its `insert` and `remove`[5] history:

$$\texttt{i}(x \mapsto 1)\texttt{@\{\}},\quad \texttt{i}(x \mapsto 2)\texttt{@\{A\}},\quad \texttt{i}(y \mapsto 3)\texttt{@\{B\}},\quad \texttt{r}(x)\texttt{@\{B\}},\quad \texttt{i}(y \mapsto 4)\texttt{@\{\}},\quad \ldots$$

$$\shortparallel$$

$$\texttt{@\{B\}}$$

$$\Downarrow$$

$$[y \mapsto 4\texttt{@\{\}}]\ \texttt{@\{B\}}$$

[5] For this particular use case, we actually only need public (`@{}`) `remove` operations.

Additions in Troupe: caching layer

Peer-to-per cache is only a tiny Troupe program that orchestrate a `HashMap` instance.

Additions in Troupe: caching layer

Peer-to-per cache is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: SHA256 `string` $\rightarrow$ 32-bit `int`. (4 LOC)

Additions in Troupe: caching layer

Peer-to-per cache is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: SHA256 `string` → 32-bit `int`. (4 LOC)
- Define and spawn a local Troupe thread that owns the `HashMap`. (21 LOC)

```
(* cache.trp : line 48 *)
HashMap.insert m auth k v
```

```
(* cache.trp : line 57 *)
HashMap.findAll m auth k
```

Additions in Troupe: caching layer

Peer-to-per cache is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: SHA256 `string` → 32-bit `int`. (4 LOC)
- Define and spawn a local Troupe thread that owns the `HashMap`. (21 LOC)

```
(* cache.trp : line 48 *)
HashMap.insert m auth k v
```

```
(* cache.trp : line 57 *)
HashMap.findAll m auth k
```

- Define and export `set` and `get` functions callable from the TCB. (8 LOC)

```
(** API Setter function *)
fun set (key, value) = send(thread, (SET, key, value))
```

Additions in Troupe: caching layer

Peer-to-per cache is only a tiny Troupe program that orchestrate a `HashMap` instance.

- Hash function for `HashMap`: SHA256 `string` $\rightarrow$ 32-bit `int`. (4 LOC)
- Define and spawn a local Troupe thread that owns the `HashMap`. (21 LOC)

```
(* cache.trp : line 48 *)
HashMap.insert m auth k v
```

```
(* cache.trp : line 57 *)
HashMap.findAll m auth k
```

- Define and export `set` and `get` functions callable from the TCB. (8 LOC)

```
(** API Setter function *)
fun set (key, value) = send(thread, (SET, key, value))
```

- Use `sandbox` builtin to guard against timing attacks. (14 LOC)

How can one add an IFC-aware cache?

How can one add an IFC-aware cache?

By bootstrapping the IFC-aware runtime!

Steffan Christ Sølvsten

soelvsten@cs.au.dk

ssoelvsten.github.io

Troupe

github.com/TroupeLang/troupe

AARHUS
UNIVERSITY

Troupe's Threat Model

Uses several security mechanisms, inspired by Jif, Fabric, and FLAM.

- Progress-sensitive Dynamic Information-flow Control
- Sandboxing
- Capabilities

Few assumptions on adversaries! But, we do ignore:

- Network Traffic Analysis. Can be avoided with via recent methods.
  (Blaabjerg and Askarov '21 & '23, Nelson, Pagnin, and Askarov '24)
- Timing-channels. Can be avoided via `sandboxing` and predictive mitigation.
  (Zhang, Askarov, Myers '10 & '12)

Additions in Troupe: caching layer (timing-attack mitigation)

Troupe provides the `sandbox`(f, t) operation to run f for t ms and either return the result, an error, or a continuation. In practice, $t = 1$ ms seems to suffice.

```
(* cache.trp : line 48 *)
val (m', t') =
    sandbox' (fn () => HashMap.insert  m auth k v) t m

(* cache.trp : line 57 *)
val (v, t') =
    sandbox' (fn () => HashMap.findAll m auth k)   t []
```

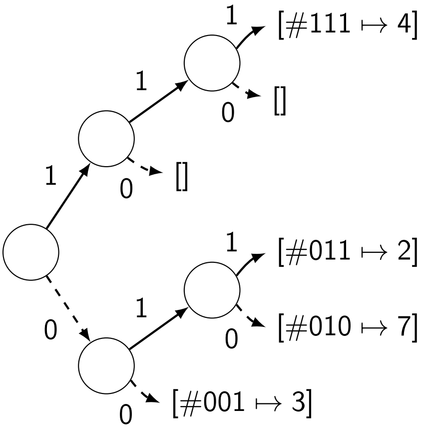
Design of the `HashMap`

Current prototype uses a Hash Array Mapped Trie (HAMT) with 200 LOC.

Only 3 declassifications per `insert`, `remove`, `find` (–24/+55 LOC):

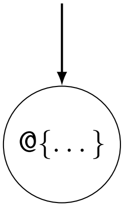
- `hash key` and `hash bucket` to not raise `pc` and taint the entire trie.
- The `bucket` list (but not its content).

But, this fully declassifies its `size`!



Design of the `HashMap`

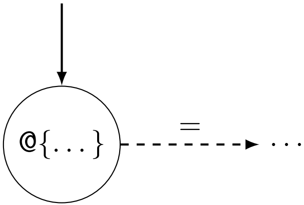
Can we design a meta data structure for `HashMap` and `Map`?



Design of the `HashMap`

Can we design a meta data structure for `HashMap` and `Map`?

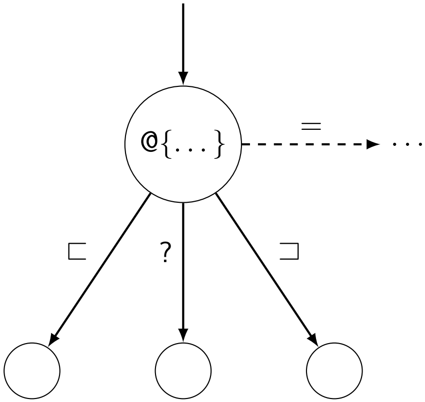
- Guard taint of secret sub structures with (first-class) IFC labels.



Design of the `HashMap`

Can we design a meta data structure for `HashMap` and `Map`?

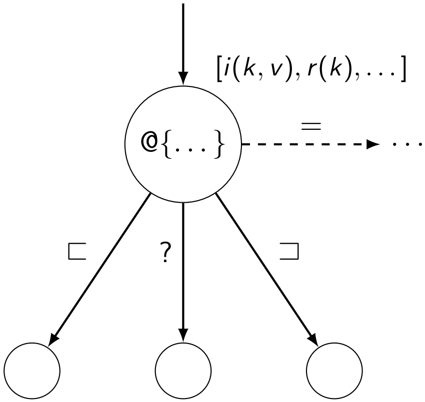
- Guard taint of secret sub structures with (first-class) IFC labels.
- Use the partial ordering on information flow ($\sqsubseteq$) to compare levels.



Design of the `HashMap`

Can we design a meta data structure for `HashMap` and `Map`?

- Guard taint of secret sub structures with (first-class) IFC labels.
- Use the partial ordering on information flow ($\sqsubseteq$) to compare levels.
- Use buffers to accumulate operations for "inaccessible" sub structures. This keeps `insert` and `remove` efficient.



Design of the `HashMap`

Can we design a meta data structure for `HashMap` and `Map`?

- Guard taint of secret sub structures with (first-class) IFC labels.
- Use the partial ordering on information flow ($\sqsubseteq$) to compare levels.
- Use buffers to accumulate operations for "inaccessible" sub structures. This keeps `insert` and `remove` efficient.
- Use time-stamps to identify latest key-value from sub structures.

But, does this make `find` too slow?